Contents

The M32 Serial Protocol 2
- Transports 4
- Receiving information from Morserino 6
- Sending commands to Morserino 6
  - Syntax of commands 6
- Enabling and disabling the M32 Protocol 6
  - Consent on BLE 7
- Versioning and Compatibility 8
- Success and Error Feedback 8
- JSON objects sent by Morserino as a result of user activities 8
  - “menu” 8
  - “control” 9
  - “config” 9
  - “activate” 9
  - “message” 9
- Objects and their GET and PUT commands 9
  - Device Information 9
  - Capabilities 10
  - Control Speed and Volume 11
  - The Menu 11
  - Configuration (Parameters) 12
  - Snapshots 14
  - Stored Text File 15
  - Other files 16
  - WiFi Configuration 16
  - Koch Lessons 17
  - Custom Character Set 18
  - Practice Set 18
  - Player Identity 19
  - Hardware Information 19
  - Battery Status 20
  - Automated CW Keyer and Memory Keyer 20
  - Practice Statistics 21
  - Game Scores 22

The M32 Serial Protocol

```
Date: August 23, 2026
Version: 1.4
Authors: Willi, OE1WKL, and Christof, OE6CHD
```

The Morserino can communicate two-way with a connected computer — over the USB bus, and (with firmware that includes the “BLE Serial” feature, and the **Bluetooth Use** preference set to **BLE Serial**) over Bluetooth Low Energy; see the section “Transports” below. Apart from keyed or generated characters (this had been implemented already previously) the Morserino can send information about user actions (selecting menus, configuring preferences etc), or about current settings etc to the computer, and the computer can send various commands to the Morserino (which enables full control over parameters and menus).

```
Changes in protocol version 1.4 from version 1.3:

New GET commands:

GET configs/details — read the full description of every parameter in a
    few pages, instead of one round trip per parameter
GET configs/details/<n> — the page of parameter descriptions that starts
    at parameter index <n>
GET capabilities — which of the build-dependent commands this firmware
    actually has, so a program need not probe for them
GET game/scores — the stored game high-score tables (builds with the
    games only)

New PUT command:

PUT game/scores/clear — clear all stored game scores and saved games,
    exactly as "Reset Scores" in the preferences menu does

Changed:

GET battery can now report status "battery" -- no external supply, running
    on the cell, and the only case where "voltage" means state of charge.
    Earlier firmware reported "charging" in that situation, which was a
    safe assumption only while the protocol required a USB cable.

GET file/list now really does list every file. Earlier firmware built the
    reply in a fixed buffer and stopped once it was full: the listing broke
    off after some 30 files, the last entry lost its "size", and the
    "total"/"used"/"free" properties never arrived at all. The shape of the
    reply is unchanged.

GET kochlesson now returns the whole character sequence in "characters",
    starting at its first character. Earlier firmware started the array at
    "minimum" instead, which mattered for exactly one sequence - LICW
    Carousel with a start of 6 or more, whose minimum is 19 rather than 1.
    There the 18 characters of the BC1 core were missing from the reply, so
    a client lining "characters" up against "value" marked the wrong ones as
    learned. "value", "minimum" and "maximum" are unchanged, and so is the
    reply for every other sequence.

Replies are now pushed out as soon as the command that produced them has
    been handled. On the M32 Pocket, whose USB is the ESP32-S3's hardware
    USB-Serial-JTAG, a short reply could otherwise stay in the transmit
    buffer until the next command arrived - so a client that waited for it
    saw nothing, and a client that gave up waiting read that stale answer as
```

```
    the reply to its NEXT command, putting every reply after it out of step.
    Intermittent, and present on every firmware before this one.

On the BLE transport, the responses that stream (GET file/list, GET
    file/text, GET file/firstline, GET stats/log) now pace themselves
    against the client instead of overrunning the device's transmit buffer,
    so they arrive whole rather than torn. See "Congestion" under
    "Transports".

Changed, on the BLE transport only:

PUT device/protocol/on now asks the Morserino's operator to confirm the
    connection on the device itself before the session begins — see
    "Consent on BLE" below. Over USB nothing changed.

Everything else is additive: no command or property from 1.3 changed its
shape or its meaning.

Additions made within protocol version 1.4, which did not change the
version number:

October 2, 2026: the Practice Set commands (GET practicechars,
PUT practicechars/set/<characters>, PUT practicechars/clear), in the
firmware since 9.0 but not described here until now, are documented
(section "Practice Set"). From firmware 9.1 the character sets of
customchars, practicechars and GET snapshot/<n> may also contain round
brackets and letters with umlauts and accents, sent and accepted as
UTF-8 (section "Custom Character Set"). Firmware 9.1 also answers
GET practicechars over BLE Serial; earlier firmware sent that reply to
USB only.

October 5, 2026: from firmware 9.1, a USB command line longer than
1024 characters is refused with the error "LINE TOO LONG" (section
"Sending commands to Morserino", "Maximum command length"). BLE has
refused lines over 400 characters since BLE Serial was introduced.
The preference "Length Words" is called "Word Length" from 9.1, with
the new value "4 or more"; GET/PUT config address it by the new name.

Additions made within protocol version 1.3, which did not change the
version number:

July 2, 2026: the "Transports" section was added — the protocol is now
also available over Bluetooth Low Energy ("BLE Serial"). No command or
property changed; BLE is a second transport for the same byte stream.

August 16, 2026: the "device" object also reports "edition" —
"standard" or "accessibility" — because the M32 Pocket's two editions
run on the same board and report the same "hardware" string. The
property is additive, and firmware that predates it simply omits it.

Changes in protocol version 1.3 from version 1.2:

New GET commands for reading device state:

GET snapshot/<n> — read the contents of a snapshot without recalling it
GET player — read player callsign and name (Fight the Pileup identity)
GET customchars — read the custom Koch character set and its active state
GET hardware — read hardware settings (brightness, left-handed mode, etc.)
GET battery — read battery / power status

New PUT commands for configuration:
```

```
PUT player/call/<callsign> — set the player callsign (stored uppercase)
PUT player/name/<name> — set the player name (stored uppercase)
PUT customchars/set/<characters> — set and enable a custom Koch character set
PUT customchars/clear — disable and clear the custom Koch character set
PUT device/reset/defaults — reset all parameters to factory defaults

Changes in protocol version 1.2 from version 1.1
   (see section "Other files"):

Three new serial protocol "put" commands for binary-safe chunked
upload (filenames can contain paths):

put file/begin/<filename> — opens the file for writing
put file/data/<base64chunk> — decodes and writes a chunk of data
put file/end — closes the file, reports the final size


Plus two utility commands to get information about  or delete a file:

get file/list — returns JSON with all files, their sizes, and space info
put file/delete/<filename> — deletes a file

And two command for handling multi-part text files:

put file/part/<number> — select a part
get file/parts — list available parts
```

Transports

The protocol is one line-based byte stream; it is available over two transports that carry identical bytes:

USB-CDC serial — as always: 115200 baud, commands end with `\n` (`\r` tolerated), responses are bare concatenated JSON objects, plus the raw character echo governed by the "Serial Output" parameter.

Bluetooth Low Energy ("BLE Serial") — available when the firmware is built with `CONFIG_BLE_SERIAL` and the device preference **Bluetooth Use** is set to **BLE Serial**. The device implements a Nordic-UART-Service (NUS) GATT server:

Item	Value
Advertised name	`Morserino-32` (carried in the scan response; scan by service UUID)
Service UUID	`6E400001-B5A3-F393-E0A9-E50E24DCCA9E`
RX characteristic (client → M32)	`6E400002-B5A3-F393-E0A9-E50E24DCCA9E`, Write and Write Without Response
TX characteristic (M32 → client)	`6E400003-B5A3-F393-E0A9-E50E24DCCA9E`, Notify

No pairing/bonding is used, but connecting is **not** by itself enough to obtain a session: because anything in radio range can connect, the operator is asked to confirm on the device (see "Consent on BLE" below). One central at a time. The M32 requests a large MTU (517); notifications are chunked to the negotiated MTU and chunk boundaries are arbitrary (they may split a JSON object or a UTF-8 character), so a client must reassemble the byte stream and frame it exactly like a USB

client would: JSON objects by balanced braces, everything outside braces is the raw character echo. Writes to RX may likewise be split or batched freely; the M32 reassembles until `\n`.

Transport rules a client should know:

- **Per-transport sessions.** Each transport has its own protocol session: `PUT device/protocol/on` on BLE starts (only) the BLE session; `PUT device/protocol/off` received over a transport ends (only) that transport's session. This command pair is matched case-insensitively on both transports.
- **Consent on BLE.** `PUT device/protocol/on` over BLE does not open a session on its own; it asks the Morserino's operator. A client must expect a `CONFIRM ON DEVICE` message and keep waiting, and must handle refusal. USB is unaffected — a cable is itself proof of presence.
- **Both sessions at once.** If both transports are handshaken, both receive all responses and unsolicited objects, including the other session's command replies (protocol 1.3 clients must tolerate unknown/unsolicited objects anyway). Session-control messages are the exception and stay on their own transport: the handshake's `device` object, the `{"end m32protocol": ...}` goodbye, BLE transport errors (`BLE LINE TOO LONG`, `BLE RX OVERFLOW`) and the BLE suspend/stop notices are delivered only to the transport they concern — a goodbye you receive always means *your* session ended. A USB port that never handshakes stays byte-silent (except the character echo and debug output, as today) even while a BLE session is active.
- **Ordering.** Commands are executed one line at a time per transport, in arrival order, with device-state changes applied between lines — batching several commands in one GATT write is fine. No ordering is guaranteed *between* the two transports within a polling cycle.
- **WiFi suspends BLE.** Any device activity that needs the WiFi radio (WiFi transceiver, multiplayer game entry, file upload, firmware update, WiFi configuration and the WiFi menu functions — including remotely executable ones like *Disp MAC Address*, so a BLE client that executes those cuts off its own link) suspends BLE Serial: the client gets a `message` object first (best effort), then the link drops. BLE Serial re-advertises when the device returns to the top menu.
- **Remote self-disable.** Setting the *Bluetooth Use* preference to any value other than *BLE Serial* via `PUT config` over BLE works: a `message` arrives best-effort, the `ok` response is lost, and the link drops.
- **Sleep suppression.** As with USB, an active protocol session suppresses the auto-power-off timeout; the suspension/disconnect paths clear this reliably, but a battery-powered device with a permanently connected, handshaken client will not auto-sleep.
- **Congestion.** Every response is paced by notify flow control. A **streamed** response — `GET file/list`, `GET file/text`, `GET file/firstline`, `GET stats/log` — is larger than the device's transmit buffer, so the device waits, briefly, for the client to take what has already been sent: such a response arrives whole as long as the client keeps reading. A client that stops reading is given up on (after about half a second), and from there the old behaviour applies: the rest of that response *and everything after it* is dropped until the backlog has fully drained. Every other response is built in memory and fits the buffer, and is never waited for — output produced on the keying path (the character echo, and objects such as `control` sent while the operator is keying) is dropped rather than delay a single dit, and the echo is always dropped before protocol replies are. If a response arrives torn or missing, re-issue the GET. **Framing after a drop:** a dropped tail leaves *unbalanced* braces in the byte stream, so a client that frames purely by brace-counting cannot resynchronize from the stream contents alone. Use a timeout: if an object stays open with no further bytes for a second or two, discard the partial object and reset the frame state

(bytes up to the next `{` are then echo, as usual). Disconnecting and reconnecting also recovers — the device fully resets the session state per connection.

Receiving information from Morserino

With the exception of keyed or generated characters (this can be controlled through the “Serial Output” parameter) the Morserino always sends a valid JSON object, delimited by curly braces (see https://de.wikipedia.org/wiki/JavaScript_Object_Notation for an explanation of JSON object notation).

Morserino sends a message whenever a user action is executed on the device (navigation within menus, executing a menu, navigation within parameter menu and setting parameters etc.). This can be used for displaying user actions on a larger screen, or for generating speech output as a feedback for all user actions (thus making the Morserino usable for blind and visually impaired persons).

Sending commands to Morserino

Commands sent to the Morserino start with “GET” (read values) or “PUT” (write values) and additional parameters. Commands are ended by a single line feed (, ascii 10); a trailing carriage return (is tolerated. Responses to GET commands are of course JSON objects, while as a rule there are no responses to PUT commands.

Maximum command length. A command line may be at most **1024 characters** over USB and **400** over BLE (bytes, including the command itself, without the line feed). A longer line is not executed: everything up to its line feed is discarded, and the Morserino answers `{"error":{"content":"LINE TOO LONG"}}` (`BLE LINE TOO LONG` over BLE) — on USB only once a protocol session is open. No command needs that much: the longest one the Morserino’s own tools send is a `PUT file/data` chunk of 240 base64 characters. If you upload a text file with `PUT file/new` / `PUT file/append`, split lines that would exceed the limit. *(USB limit from firmware 9.1; earlier firmware accepted any length, and the M32 Pocket could lose bytes of a USB command longer than about 256 characters.)*

Syntax of commands

GET *object*

GET *object/specifier*

PUT *object/specifier/value*

GET and PUT are separated by a blank from the following parts of the command, while object, specifier and value are separated by a slash (/).

GET, PUT, *object* and *specifier* can be upper or lower case, but value is case sensitive.

Enabling and disabling the M32 Protocol

After establishing the connection physically, the M32 protocol needs to be enabled on the Morserino. This is done with the following command:

`PUT device/protocol/on`

As a confirmation device information is returned, e.g.: `{"device":{"hardware":"M32 2nd edition","firmware":"8.1","protocol":"1.3","build":"Jun 22 2026","edition":"standard"}}`

From this response the connected computer program gets not only confirmation that the communication has been established, but also information about the hardware used, as well as the firmware and protocol versions.

While the protocol is enabled, the time-out of the connected Morserino is disabled, i.e. it will not go into sleep modus.

The protocol can also be disabled with the command: `PUT device/protocol/off`

The device acknowledges this with a farewell object: `{"end m32protocol":{"content":"Goodbye!"}}`.

Consent on BLE

Over USB the cable is the authorisation: whoever can plug it in can already do anything to the device. Over Bluetooth that is not true — anything within radio range can connect — so `PUT device/protocol/on` received over BLE does not open a session by itself. It asks the Morserino's operator, and the session begins only if they agree. **Over USB none of this applies** and the handshake behaves exactly as before.

A BLE client must therefore be ready for four replies instead of one. All of them are sent to the BLE session alone:

- `{"message":{"content":"CONFIRM ON DEVICE"}}` — the device is now asking its operator. Tell the user to press **FN** (the red button) on the Morserino, and go on waiting: the next object is the answer. Allow about 20 seconds; the device refuses if nobody answers in that time.
- `{"device":{ … }}` — allowed. This is the ordinary handshake response, just later than usual, and the session is open.
- `{"error":{"content":"CONNECTION DECLINED"}}` — the operator refused, or nobody answered in time.
- `{"error":{"content":"DEVICE BUSY"}}` — the Morserino is running an interactive mode, and a request is only ever put to the operator at the top menu, so it was refused without asking. Tell the user to return the device to its main menu and try again.

Once consent has been given, a client reconnecting within **60 seconds** is admitted without the operator being asked again. That is what lets a dropped link, or an app that was merely backgrounded, resume without troubling them; the window also applies while an interactive mode is running, so a session that drops mid-practice can restore itself.

Older clients are not locked out. A client written before this existed will usually treat the `CONFIRM ON DEVICE` message as a failed handshake and disconnect. The Morserino keeps the question on its screen regardless of what the client does, so the operator can still press FN — and the client's next connection attempt is then admitted through the 60-second window. The effect is one extra connect attempt, not a lockout.

Versioning and Compatibility

The protocol version is reported in the `device` object as the `protocol` property (currently “1.4”). Changes within a major version are **additive**: new GET/PUT commands and new properties may be added, but existing ones keep their meaning. A connected program should therefore **ignore any object or property it does not recognise**, and treat an unknown command (one that returns an “UNKNOWN COMMAND” or “NOT YET IMPLEMENTED” error) as “not supported by this firmware”.

To identify the firmware it is talking to, a program uses the `device` object: `firmware` (the firmware version), `protocol` (the protocol version), `build` (the firmware compile date), and `edition` (which firmware edition is running).

Not every command exists in every firmware, because some features are compiled out of some builds — the games, for instance, exist only on the M32 Pocket. Since version 1.4 a program can ask instead of probing: `GET capabilities` lists the build-dependent commands the connected firmware actually has (see “Capabilities” below). Commands that are *not* build-dependent are implied by the reported protocol version and are not listed there.

A program that must also work with firmware older than 1.4 should fall back gracefully: if `GET capabilities` returns an error, the firmware predates 1.4, and the program can use the 1.3 commands and treat every error reply as “not supported”.

Success and Error Feedback

When the M32 could parse and execute the command, an “ok” object is returned, e.g.

`{“ok”:{“content”:“OK”}}.

Exceptions are the `PUT control` commands: they return the “control” objects, in the same way as `GET control`.

When invalid commands are sent to the Morserino, an “error” object is returned, with the error text in the “content” property, e.g.

`{"error":{"content":"INVALID Value xxx"}}` .

JSON objects sent by Morserino as a result of user activities

“menu”

As a result of menu navigation, a *menu* object is sent with the following properties:

- “content” (type String): The complete menu path, with elements separated by slash (/).
- “menu number” (type Number): A number to identify this menu item.
- “executable” (type Boolean): If false, there are sub-menu items, if true this can be executed.
- “active” (type Boolean): Always false when resulting from user activity.

Example:

```
{"menu":{"content":"CW Generator/..",
"menu number":2,"executable":false,"active":false}}
```

“control”

As a result of changing the speed of the keyer, or the audio volume, a control object is sent with the following properties:

- “name” (type String): “speed” or “volume”.
- “value” (type Number): For “speed”: keyer speed in words per minute, for “volume” a number between 0 and 19.

Example:

```
`{"control":{"name":"speed","value":16}}`
```

“config”

As a result of navigating the parameters menu, a config object is sent with the following properties:

- “name” (type String): The name of the currently selected parameter.
- “value” (type Number): The current value, as stored internally, as a number.
- “displayed” (type String): How the value is displayed (often as a meaningful text).

Example:

```
{"config":{"name":"External Pol.","value":0,"displayed":"Normal"}}
```

“activate”

As a result of activating a menu, an activate object is sent with the following property:

- “state” (type String): can be any of “EXIT”, “ON”, “SET”, “CANCELLED”, “RECALLED”, “CLEARED”

Example:

```
{"activate":{"state":"ON"}}
```

“message”

Sometimes, as a result of some user action, a message is displayed on the screen. In these cases a message object is sent with the following property:

- “content” (type String): The message body.

Example:

```
{"message":{"content":"Generator Start / Stop press Paddle  "}}
```

Objects and their GET and PUT commands

Device Information

`GET device` Returns the properties „hardware“ (e.g. “M32 1st edition”, “M32 2nd edition”, or a board name such as the M32 Pocket’s HW_NAME) „firmware“ (the firmware version number), „protocol“ (the M32 Protocol version), „build“ (the firmware compile date, e.g. “Jun 22 2026”), and „edition“ (which firmware edition is running: “standard”, or “accessibility” for the build that speaks the user interface aloud).

„edition“ exists because the hardware string does not identify the firmware on its own: the M32 Pocket's standard and Accessibility editions run on the same board and both report „hardware“ as “M32 Pocket (Wroom)”. A program that has to tell them apart — to offer the matching user manual, for instance — reads „edition“. Firmware released before August 2026 omits the property; treat its absence as “not known” rather than as “standard”.

Example:

```
{"device":{"hardware":"M32 2nd edition","firmware":"9.0","protocol":"1.4","build":"Aug
19 2026","edition":"standard"}}
```

`PUT device/protocol/on`
This switches the M32 protocol on (you will get device information back; you will also get device info when command is sent while protocol is already ON); while this is on, there will be no time-outs, whatever the parameter says!

`PUT device/protocol/off`
This switches the M32 Protocol off (time-out as defined in its parameter will be in effect again).

`PUT device/reset/defaults` This resets all configurable parameters to their factory default values. Snapshots, WiFi configuration, CW memories, and player identity are not affected.

The reset can only be carried out during start-up (before the stored values are loaded), so the device acknowledges the command, then **reboots** to perform it. The reboot switches the M32 protocol off and drops the serial connection; the connected program must reconnect and send `PUT device/protocol/on` again to continue.

Capabilities

`GET capabilities`

(protocol version 1.4)

Returns the properties “protocol” (the protocol version, the same value as in the `device` object) and “features” (type Array of Strings): the **build-dependent** commands this particular firmware has.

Example:

```
{"capabilities":{"protocol":"1.4",
"features":["configs/details","game/scores","stats/log"]}}
```

Only commands that some builds *lack* appear in “features”; everything else the protocol version documents is always present. Today that list can contain:

- `configs/details` — the bulk parameter read (present in every 1.4 firmware),
- `game/scores` — the game high-score commands (builds with the games, i.e. the M32 Pocket),
- `stats/log` — the practice-statistics log (builds with practice statistics).

A program written for 1.4 or later should call this once after `PUT device/protocol/on` instead of probing commands and reading error replies. Firmware older than 1.4 answers `GET capabilities` with an error — that itself identifies it as pre-1.4.

Note that the *parameter* set is build-dependent in the same way, but it is not described here: read it from `GET configs` (or `GET configs/details`), which reports what the device actually has.

Control Speed and Volume

`GET control/speed`

This returns the properties “name”, “value”, “minimum” and “maximum” for keyer speed (in words per minute - wpm.

`GET control/volume`

This returns the properties “name”, “value”, “minimum” and “maximum” for the volume control.

`GET controls`

This returns both controls and their current values in a list (without minimum/maximum), e.g. `{"controls":[{"name":"speed","value":18},{"name":"volume","value":12}]}`.

`PUT control/speed/<value>`

Use this to set the keyer speed to <value> (numeric value, words per minute).

`PUT control/volume/<value>`

Use this to set the speaker volume to <value> (numeric value, 0 = no sound, 19 = maximum volume).

The Menu

`GET menus`

This returns a list of all menu items, with the properties “content” (a string giving the complete menu path), “menu number” (a unique number for each menu item), and „executable“ (a boolean value that indicates if this can be executed - if “true” - or it has sub-menu items - if “false”).

Example:

```
{
"menus": [
    {
        "content": "CW Keyer",
        "menu number": 1,
        "executable": true
    },
    {
        "content": "CW Generator/..",
        "menu number": 2,
        "executable": false
    },
    {
        "content": "CW Generator/Random",
        "menu number": 3,
        "executable": true
    }, ...
 ]}
```

`GET menu`

Returns the current menu object with properties “content” (= menu path), “menu number”, “executable” and “active”; if called while a mode (eg CW Keyer) is running, (and not just selected in the menu), the property “active” shows “true”, otherweise “false”.

Example:

```
{"menu":{"content":"CW Generator/English Words",
"menu number":5,"executable":true,"active":true}}
```

`PUT menu/set/<number>`

Set the main menu to the requested menu entry (even if it is not executable!). You have to use the menu number.

`PUT menu/start`

If in menu mode, start the currently selected menu, if it is executable (do nothing when not in menu mode, or when the current menu entry is not executable).

`PUT menu/start/<menu number>`

Start the command that has the number `<menu number>` (only if it is executable!).

For this, the M32 must be in the main menu (if not sure, execute put menu/stop before doing this, or check with `GET menu` if the "active" property is "false")

`PUT menu/start now`

`PUT menu/start now/<menu number>`

These commands perform basically the same task as `PUT menu/start` and `PUT menu/start/<menu number>` ; there is a difference only when starting one of the CW Generator modes: normally these modes, after starting them, wait for a paddle (or key) action to get going. Startimng them with any these commands will get them going immediately, without waiting that the user presses a key or touches a paddle.

`PUT menu/stop`

If M32 is active or in the preferences menu, stop it and go to main menu. At the main menu it does nothing (firmware before 9.2 kept it pending, and the next mode started then ended at once).

Configuration (Parameters)

`GET configs`

This returns a list of all configurable parameters, with the properties "name" (= parameter name), "value" (as numerical values), and "displayed" (how the value has to be displayed).

Example:

```
{"configs":[{"name":"Encoder Click","value":1,"displayed":"On"},
{"name":"Tone Pitch","value":10,"displayed":"622 Hz e2"},
{"name":"External Pol.","value":0,"displayed":"Normal"},...
]}
```

The exact set of parameters depends on the hardware variant and firmware build (for example "Headphone Output" and "Theme" exist only on the M32 Pocket, "Invader Orient." only on builds with the games). A program that restores a configuration by parameter name should match case-insensitively and **tolerate an "INVALID PARAMETER" error** for any parameter not present on the connected device, rather than aborting the whole restore.

`GET config/<parameter name>`

This returns all details of that parameter (or an error if an invalid parameter name has been given; upper and lower case in parameter names are not significant). The returned properties are:

- "name" (type String): the name of the parameter,

- “value” (type Number): the current value of the parameter,
- “default” (type Number): the value the parameter has after a factory reset (*firmware V9 and later*; older firmware omits the property entirely, so a client must treat its absence as “not known” rather than assume a value), “description” (type String): a more verbuous description what the parameter is about,
- “minimum” (type Number),
- “maximum” (type Number),
- “step“ (type Number),
- “isMapped“ (type Boolean):”true” if the value has to be shown as some character string, and
- “mapped values” (type Array of Strings): for the value to text mappings, if applicable

Example:

```
{"config":{"name":"Keyer Mode","value":2,"default":1,
"description":"Iambic Modes, Non-squeeze mode, Straight Key mode",
"minimum":1,"maximum":5,"step":1,"isMapped":true,
"mapped values":["","Iambic A","Iambic B","Ultimatic",
"Non-Squeeze","Straight Key"]}}
```

`PUT config/<parameter name>/<value>`

This sets the named parameter to the specified value - the value must be the NUMERIC value, and not the textual representation!

Example to set Keyer Mode to Iambic B:

```
PUT config/keyer mode/2
```

`GET configs/details` `GET configs/details/<n>`

(protocol version 1.4)

This returns the **full details of many parameters at once** — the same information `GET config/<parameter name>` gives, but for a whole page of parameters per command instead of one. A program that builds a preferences view needed one round trip per parameter before; with some fifty parameters that is several seconds over USB and about ten over BLE, and this reduces it to a handful of commands.

The reply is one page. `GET configs/details` returns the first page; `GET configs/details/<n>` returns the page that starts at parameter index `<n>`. The properties are:

- “from” (type Number): the parameter index this page starts at,
- “count” (type Number): how many parameters this page contains,
- “total” (type Number): how many parameters the device has altogether — advisory, useful for a progress display,
- “more” (type Boolean): “true” if there are further pages,
- “items” (type Array of Objects): the parameters themselves. **Each item is exactly what `GET config/<parameter name>` returns**, without its enclosing `{"config": … }`.

To read them all, start at `GET configs/details` and, as long as “more” is true, ask for the next page at index “from” + “count”.

Example:

```
GET configs/details
-> {"configdetails":{"from":0,"count":8,"total":49,"more":true,
```

```
"items":[{"name":"Keyer Mode","value":2,"default":1,
"description":"Iambic Modes, Non-squeeze mode, Straight Key mode",
"minimum":1,"maximum":5,"step":1,"isMapped":true,
"mapped values":["","Iambic A","Iambic B","Ultimatic",
"Non-Squeeze","Straight Key"]}, … ]}}

GET configs/details/8
-> {"configdetails":{"from":8,"count":8,"total":49,"more":true, … }}

GET configs/details/48
-> {"configdetails":{"from":48,"count":1,"total":49,"more":false, … }}
```

The parameters come in the same order, and are the same set, as in `GET configs` — so index `<n>` means the same parameter in both commands.

Notes:

- **The device chooses the page size, not the program.** Always take the next page from "from" + "count", and stop when "more" is false; never assume a fixed number of items per page, because the device may return fewer.
- "total" depends on the hardware variant and the firmware build, exactly as the parameter set itself does.
- An out-of-range `<n>` returns `{"error":{"content":"INVALID PARAMETER"}}` rather than an empty page, so that a program which has lost its place finds out.

Snapshots

`GET snapshots`

This command returns a list of all currently stored snapshots.

Example:

```
{"snapshots":{"existing snapshots":[1,2,3]}}
```

`PUT snapshot/store/<n>`

Store the current parameters (and the currently selected menu item) in snapshot n (n = 1..8). Since firmware 8.2 the device verifies that the snapshot actually reached non-volatile storage: on success it answers with `{"ok":{"content":"OK"}}`; if the settings storage is full, the store fails with

```
{"error":{"content":"SNAPSHOT STORE FAILED – NVS FULL?"}}
```

and the list of stored snapshots remains unchanged. Space can be freed by clearing a snapshot or resetting the game scores.

`PUT snapshot/recall/<n>`

Recall parameters (and menu selection) from snapshot n.

`PUT snapshot/clear/<n>`

Clear (i.e., delete) snapshot n (n = 1..8).

`GET snapshot/<n>`

This command returns the full contents of snapshot n (n = 1..8) without recalling it — the current device settings are not modified. This is useful for inspecting or comparing snapshot contents. The snapshot must exist (see `GET snapshots` to check which snapshots are stored).

The response includes the stored menu selection, custom character set, and all parameter values with their display representations. Note that since firmware 8.2 snapshots contain only training-related settings: device, hardware, connectivity and game settings (paddle polarity, external TX keying, Generator Tx, LoRa channel, Bluetooth Use, audio routing, Encoder Click, Quick Start, Invader orientation, and the QSO Bot settings) are not stored in snapshots — like the “Serial Output” and “Time-out” parameters, they are absent from the `configs` list. Snapshots written by older firmware may still contain such entries; they are ignored on recall and dropped when the snapshot is converted or overwritten.

Example:

```
{"snapshot":{"number":3,"lastExecuted":1,"menuName":"CW Keyer",
"customChars":{"active":false,"characters":""},
"configs":[
{"name":"Tone Pitch","value":10,"displayed":"622 Hz e2"},
{"name":"Keyer Mode","value":2,"displayed":"Iambic B"},
{"name":"InterWord Spc","value":7,"displayed":"7"},
...
]}}
```

Stored Text File

`GET file/size`

This command return some usage statistics of the SPIFFS file system (used for the file the file player can play), with properties “size” (type Number; the size of the current player file in bytes) and “free” (type Number; the size of the available storage in bytes. Be aware that the actual free space might be a little bit less than indicated, because file space is allocated in chunks).

Example:

```
{"file":{"size":21,"free":1498219}}
```

`GET file/text`

This command returns the contents of the text file used by the file player, in an object named “file”, as a property (type String) called “text”.

Example:

```
{"file":{"text":"The first line of a small text file.\n
The second line, with a pause at the end. <p>\n"}}
```

`GET file/first line`

This returns the contents of the first line of the text file used by the file player, in an object named “file”, as a property (type String) called “first line”.

If you use the comment feature for text files (a line beginning with “<c>” or “/c” is regarded as a comment and not played), you can use this to give you some indication about the file contents.

Example:

```
{"file":{"first line":"<c> This file contains sample QSO text."}}
```

`GET file` This is just a short version of `GET file/first line`.

`PUT file/new/<line of text>`

This command replaces an existing existing file with a new file, and place a line of text into the new file.

`PUT file/append/<line of text>`

This will append a line of text to the existing file. By using `PUT file/new/<line of text>` and then a series of `PUT file/append/<line of text>` commands you can upload a text file with many lines of text.

`GET file/parts`

This command will list available parts within the text file (see the User Manual how multi-part text files are defined how to create them).

`PUT file/part/<number>`

With this command you can select a part of a multi-part file to be used by the file player.

Other files

The protocol now also supports file upload of arbitrary files into the SPIFFS file system, also binary files, encoded as base64. This can be used, for example to upload mp3 files as replacements for the success and error sounds of the Echo Trainer. All file names can contain a preceding path.

`PUT file/begin/<filename>`

This opens the file for writing (any filename, including a path).

`PUT file/data/<base64chunk>`

This decodes and writes a chunk of base64-encoded data. A single chunk may decode to **at most 256 bytes** of binary data (i.e. up to about 344 base64 characters); the host tools use 180 bytes per chunk. A chunk that decodes to more than 256 bytes is rejected with an error and not written.

`PUT file/end`

This closes the file, and reports the final size.

`GET file/list`

This command returns a JSON with all files, their sizes, and SPIFFS space info:

```
{"files":[{"name":"/player.txt","size":14233},
{"name":"/sounds/success.mp3","size":8102}],
"total":4718592,"used":4468122,"free":250470}
```

The reply is streamed and lists **every** file on the device. On the Accessibility Edition that includes the roughly 500 speech clips in `/voice/`, which makes for a reply of about 20 KB: a program should allow for that in its read timeout, and is well advised to keep those clips out of the file list it shows the user (they belong to the firmware, not to the operator).

`PUT file/delete/<filename>`

This command deletes a file from the SPIFFS file system.

WiFi Configuration

`GET wifi`

This command will return the currently defined wifi entries (the properties “ssid” and “trxpeer”, but NOT “password” (!), and if the property “selected” (type Boolean) is true, this entry will be used

selected for connection. There is now also a field that indicates if EspNow is selected for WiFi (this is only true when selected is false for all other WiFi options):

```
{"wifi":[{"ssid":"...","trxpeer":"...","selected":false}, ...], "espnow":true,
"wlanChoice":0}
```

Example:

```
{"wifi":[{"ssid":"Shackrouter","trxpeer":"cq.morserino.info","selected":false},
{"ssid":"MyHomeRouter","trxpeer":"cq.morserino.info","selected":true},
{"ssid":"","trxpeer":"","selected":false}]}
```

`PUT wifi/ssid/<n>/<ssid>`

This sets the SSID for WiFi setting <n> (n = 1..3).

`PUT wifi/password/<n>/<password>`

This sets the password for WiFi setting <n>. For security the password is **write-only**: it can be set, but is never returned by `GET wifi` (and therefore cannot be included in a backup).

`PUT wifi/trxpeer/<n>/<trxpeer>`

This sets the entry for TRX Peer for WiFi setting <n>.

`PUT wifi/select/<n>`

This selects which connection to use: <n> = 1, 2 or 3 selects one of the three WiFi entries, while **<n> = 0 selects EspNow** (peer-to-peer WiFi without an access point).

Koch Lessons

`GET kochlesson`

This returns the currently selected Koch lesson, with properties “value” (type Number; the numeric value of the currently set Koch lesson), “minimum” (type Number), “maximum” (type Number), and “characters” (type Array of Strings; thee are the characters in the currently selected order of characters (parameter “Koch Sequence”).

Example:

```
{"kochlesson":{"value":9,"minimum":1,"maximum":51,"characters":
["m","k","r","s","u","a","p","t","l","o","w","i",".","n","j","e","f",
"0","y","v",",","g","5","/","q","9","z","h","3","8","b","?","4","2",
"7","c","1","d","6","x","-","=","<sk>","+","<as>","<kn>","<ka>",
"<ve>","<bk>","@",":"]}}
```

`PUT kochlesson/<n>`

This sets Koch lesson to lesson number <n>.

Note: “characters” always lists the whole character sequence, from its first character, and not only the part already learned — “value” says how far into it the current lesson reaches, so a client can show the sequence entire and mark the first “value” characters as learned. When “Koch Sequence” is set to “Custom Chars” the lesson indexes into the custom character set instead: “maximum” and “characters” then describe that set rather than the fixed 51, and the active training pool for the CW Generator / Echo Trainer is its first “value” characters.

Custom Character Set

`GET customchars`

This returns the current state of the custom Koch character set, with properties “active” (type Boolean; whether custom characters are enabled) and “characters” (type String; the character set, empty if not defined).

Example:

```
{"customchars":{"active":true,"characters":"mkrsuaptl"}}
```

`PUT customchars/set/<characters>`

This sets and enables a custom Koch character set. The `<characters>` string contains the characters to be used, in the desired order. Note: `<characters>` is case-sensitive in the protocol (it is the third argument, which is not lowercased by the command parser).

Character sets in this command, in `GET customchars`, in `GET snapshot/<n>` and in the Practice Set commands below use the device’s own one-character codes: lower-case letters, digits and punctuation as themselves, and the prosigns as upper-case letters (`S` = <as>, `A` = <ka>, `N` = <kn>, `K` = <sk>, `E` = <ve>, `B` = <bk>, `H` = ch). From firmware 9.1 a set may also contain the round brackets and the letters with umlauts and accents that have a Morse code – ä ö ü é è à ç ñ å æ ø – written as ordinary UTF-8 characters, both ways; an accented capital (É) is stored as its small letter, and accented letters without a Morse code (á, ê, …) are stored as the plain letter.

Example:

```
PUT customchars/set/mkrsuaptlowi
```

Note: on the device, the custom character set can also be derived from the uploaded player file (`/player.txt`). A set injected with this command persists across preferences-menu visits and stays active until explicitly replaced — it is *not* overwritten by exiting the preferences menu. It is only re-derived from the player file when “Koch Sequence” is explicitly (re-)selected to “Custom Chars” (via the on-device menu, or via `PUT config/koch sequence/4`), so uploading a new player file and then re-selecting “Custom Chars” is how you load its content, overwriting anything injected with this command.

`PUT customchars/clear`

This disables and clears the custom Koch character set. The Koch trainer will revert to the sequence selected by the “Koch Sequence” parameter.

Practice Set

`GET practicechars`

This returns the Practice Set – the character pool of the **Practice Set** entries of CW Generator and Echo Trainer, and of **Boost Practice** – in property “characters” (type String, empty if not set), in the same one-character codes as the custom character set above.

Example:

```
{"practicechars":{"characters":"kmré"}}
```

`PUT practicechars/set/<characters>`

This sets the Practice Set (at most 24 characters; it is stored immediately and is not part of any snapshot). `<characters>` is case-sensitive, as for `customchars`.

`PUT practicechars/clear`

This clears the Practice Set.

Player Identity

These commands read and set the player callsign and name used in the “Fight the Pileup” game. Both callsign and name are stored as uppercase, with a maximum length of 8 characters.

`GET player`

This returns the stored player callsign and name.

Example:

```
{"player":{"call":"OE1WKL","name":"WILLI"}}
```

`PUT player/call/<callsign>`

This sets the player callsign. The value is converted to uppercase and truncated to 8 characters.

Example:

```
PUT player/call/OE1WKL
```

`PUT player/name/<name>`

This sets the player name. The value is converted to uppercase and truncated to 8 characters.

Example:

```
PUT player/name/Willi
```

Hardware Information

`GET hardware`

This returns read-only information about hardware settings. These values are displayed for informational purposes and cannot be changed via the serial protocol (so a backup tool can read them for reference, but cannot restore them).

The properties are: “brightness” (type Number; OLED brightness level), “leftHanded” (type Boolean; whether screen is flipped for left-handed use), and “vAdjust” (type Number; battery voltage calibration value). On the M32 Pocket the response also includes “cn3Paddle” (type String; “touch” or “mechanical” — the paddle type configured for the CN3 connector). On devices with LoRa hardware, the response also includes “loraBand” (type Number; 0=433, 1=868, 2=920 MHz), “loraFrequency” (type Number; exact frequency in Hz), and “loraPower” (type Number; transmit power in dBm).

Example (M32 with LoRa):

```
{"hardware":{"brightness":255,"leftHanded":false,"vAdjust":210,
"loraBand":0,"loraFrequency":434150000,"loraPower":14}}
```

Example (M32 Pocket, no LoRa):

```
{"hardware":{"brightness":255,"leftHanded":false,"vAdjust":210}}
```

Battery Status

`GET battery`

This returns information about the power / battery status.

On devices with battery monitoring hardware (e.g. M32 Pocket), the response includes “voltage” (type Number; millivolts) and “status” (type String). The status is read from the charge-controller status pins — the same source the device’s own battery icon uses, *not* a voltage threshold — and is one of:

- `"battery"` — no external supply: the device is running off the cell. **Here, and only here, is “voltage” a meaningful state of charge.**
- `"charging"` — an external supply is present and the cell is charging.
- `"full"` — an external supply is present and charging is complete.
- `"no battery"` — an external supply is present and no cell is installed.

While anything other than `"battery"` is reported, the voltage sits near 4.2 V almost immediately and says nothing about charge level: use “status”, not “voltage”, to tell charging from full.

`"battery"` exists because BLE removed the assumption that a client implies a cable. Firmware that predates it reports `"charging"` when running on the cell — a client that must work with older firmware should treat “charging” as “powered, state unknown”.

On devices without battery monitoring (classic M32) the status is always `"usb powered"`: there is no power-good signal to read, so a classic M32 running on its battery cannot be distinguished from one on USB.

Example (M32 Pocket):

```
{"battery":{"voltage":4050,"status":"charging"}}
```

Example (M32 Pocket, no cable connected):

```
{"battery":{"voltage":3870,"status":"battery"}}
```

Example (classic M32):

```
{"battery":{"status":"usb powered"}}
```

Automated CW Keyer and Memory Keyer

`PUT cw/play/<text to be played>`

This command plays the text string in CW (like a memory keyer - needs to be in CW Keyer mode or in Morse Trx mode, cannot work in LoRa or WiFi Trx modes). Keying will be stopped as soon as something is being keyed manually, or with the `PUT cw/stop` command.

`PUT cw/repeat/<text to be played>`

This is similar to the command above, but the text string is repeated until stopped.

`PUT cw/stop`

This will stop the CW player (it has the same effect as beginning to key manually while the commands `PUT cw/play` or `PUT cw/repeat` are active).

Note: you can use “<p>” or “” within `<text to be played>` to generate a short pause.

`PUT cw/store/<n>/<content>`

Store `<content>` in permanent memory number `<n>` (n is 1 .. 8); if `<content>` is an empty string, this memory is deleted. `<content>` can be normal Morse code characters, pro signs, e.g. “<bk>”, and also “[p]” or “” for a pause.

`PUT cw/recall/<n>`

Generate Morse code from the content in memory number `<n>`; if `<n>` is 1 or 2, do this until stopped by touching a paddle, or by the `PUT cw/stop` command.

`GET cw/memories`

Get a list of memory numbers that have some content stored.

Example:

```
get cw/memories
-> {„CW Memories":{"cw memories in use":[1,3,4]}}
```

`GET cw/memory/<n>`

Get contents of memory number `<n>`.

Example:

```
get cw/memory/
-> {"CW Memory":{"number":1,"content":"cq cq cq de oe1wkl oe1wkl [p]"}}
```

Note: Content stored in permanent memory can be recalled (“played”) in Morserino-32’s keyer mode, even when the Morserino is operated stand-alone and there is no serial connection to a computer (see user manual for details).

Practice Statistics

(only on firmware that includes practice statistics — check for `stats/log` in `GET capabilities`)

The device can keep a log of practice sessions in its file system (`/stats.jsonl`, one JSON object per line).

`GET stats/log`

Returns the properties “log” (type String): the whole log file, **Base64-encoded**; “used” (type Number): bytes used by the entire file system, not just this log; “total” (type Number): the file system’s size; “logSize” (type Number): the size of the log file itself; and “enabled” (type Boolean): whether logging is currently switched on.

The log is Base64-encoded rather than sent as text because its raw content is itself full of `{` and `}` — a program that finds the end of a reply by counting braces would lose its place (see “Syntax of commands”). Decode “log” to get the JSON-lines text.

Example:

```
{"stats":{"log":"eyJ0IjoxNzUw…","used":198412,"total":1310720,
"logSize":4096,"enabled":true}}
```

Before answering, the device closes any practice segment still in progress, so a session that is under way at that moment is included rather than missing.

`PUT stats/clear`

Deletes the log.

`PUT stats/enabled/<value>`

Switches logging on (“1” or “true”) or off (any other value).

Game Scores

(protocol version 1.4; only on firmware that includes the games — check for `game/scores` in `GET capabilities`)

`GET game/scores`

Returns the stored high-score tables of all games, and whether Radio Cave has a saved game. Each entry in “games” has:

- “id” (type String): a stable name for the game, safe to match on — “invaders”, “morsel”, “trailblazer”, “foxhunt”, “memorychain-chars”, “memorychain-calls”, “radiocave”,
- “name” (type String): the game’s name as shown on the device,
- “scores” (type Array of Objects): the stored rows, best first. Empty table entries are left out, so a game that was never played returns an empty array.

The fields of a score row differ from game to game, because the games are scored differently:

Game	Fields of one score row
Morse Invaders	“score”, “koch” (Koch lesson), “level”, “wpm”
Morsel	“time” (adjusted time in ms, lower is better), “guesses”, “length” (word-length setting), “koch”, “solved”, “total”
Trailblazer, Fox Hunt	“cpm” (effective characters per minute, the ranking key), “time” (raw solve time in ms), “wrong”, “steps”, “koch”
Memory Chain (Characters)	“chain” (chain length reached), “boxes”, “errors”, “koch”, “prompt” (“Display” or “Sound”)
Memory Chain (Call Signs)	“calls”, “letters”, “prompt”
Radio Cave	no score table; the entry carries “saved” (type Boolean) instead

Example:

```
{"gamescores":{"games":[
{"id":"invaders","name":"Morse Invaders",
 "scores":[{"score":2480,"koch":21,"level":3,"wpm":18}]},
{"id":"morsel","name":"Morsel","scores":[]},
{"id":"trailblazer","name":"Trailblazer",
 "scores":[{"cpm":37,"time":29400,"wrong":1,"steps":18,"koch":21}]},
{"id":"foxhunt","name":"Fox Hunt","scores":[]},
{"id":"memorychain-chars","name":"Memory Chain (Characters)","scores":[]},
{"id":"memorychain-calls","name":"Memory Chain (Call Signs)","scores":[]},
{"id":"radiocave","name":"Radio Cave","scores":[],"saved":true}
]}}
```

`PUT game/scores/clear`

Clears **all** game high-score tables and the Radio Cave saved game — exactly what “Reset Scores” in the preferences menu does, and just as irreversibly. Morsel’s word-length setting and Memory Chain’s lobby settings are preferences, not scores, and are kept.

```
put game/scores/clear
-> {"ok":{"content":"OK"}}
```

While a game is actually being played the command is refused with `{"error":{"content":"GAME IN PROGRESS"}}` — a game holds its table in memory and writes it out again at the end of a round, so a wipe at that moment would simply reappear. Leave the game first.

Note: scores can be read and cleared, but **not written**. There is deliberately no command to store a score, so a connected program cannot invent one.